UNIVERSIDAD TECNOLÓGICA METROPOLITANA

DESARROLLO Y GESTIÓN DE SOFTWARE

OPTATIVA

QUINTAL WARNER MICHELLE ALESSANDRA

Tarea 3

INTEGRANTES:

Buenfil Yunes Julián Nahim

López Uicab Rossana Sofía

Martinez Rivero Allan Alexis

Sima Moo Jafet de Jesús

10° Cuatrimestre

2° Parcial

10-B

27 de octubre de 2026

REPO: https://github.com/Jess4lxy/StockAlert-project.git

1. Indicadores

Se definieron tres métricas a partir de los 10,000 movimientos cargados en la Tarea 2. Cada una responde a una pregunta que el dueño o el encargado de almacén de un negocio pequeño se hace todos los días. El dashboard las calcula sobre los últimos 3, 6 o 12 meses calendario hasta el último movimiento registrado; los valores de la tabla corresponden a los últimos 6 meses (mayo a octubre de 2026).

Métrica	Cómo se calcula	Decisión que apoya	Valor (6 meses)
Productos bajo su stock mínimo	Productos activos con stock_actual menor que stock_minimo	Qué productos comprar hoy y cuántas unidades faltan de cada uno	4 de 93 productos
Unidades vendidas en el periodo	Suma de las salidas del periodo, comparada con el mismo número de meses anteriores	Cuánto presupuesto de compras planear y si las entradas siguen el ritmo de las ventas	14,412 unidades (−2.8 %)
Categoría más vendida	Categoría con más unidades vendidas en el periodo y su participación en el total	A qué categorías dar prioridad de surtido, espacio en anaquel y negociación con proveedores	Papelería, 31.3 % (4,507 unidades)

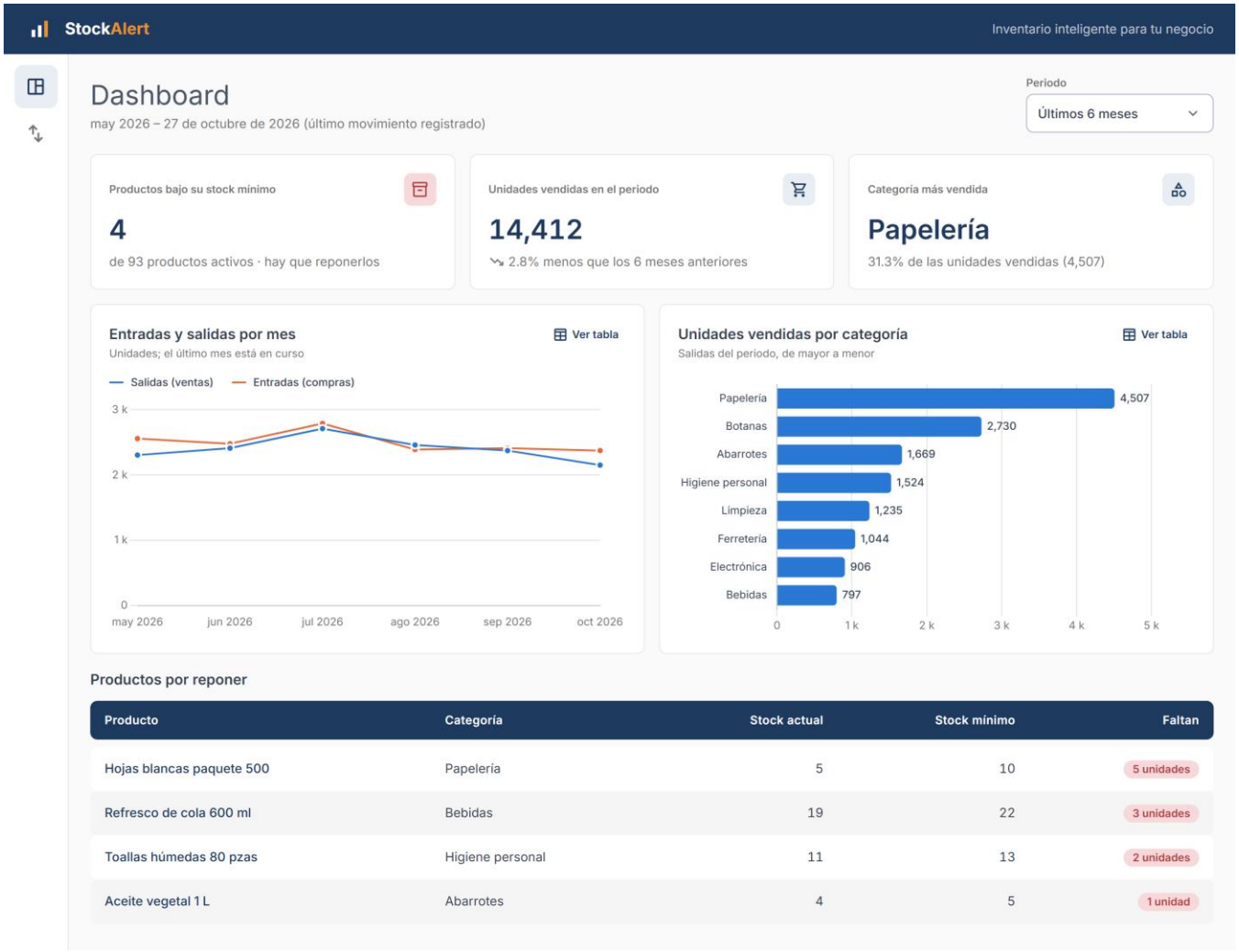
2. Dashboard

El dashboard es la pantalla de inicio de la aplicación web (Vue 3). Se alimenta de un solo endpoint, GET /api/dashboard?meses=3|6|12, que hace las agregaciones en PostgreSQL y devuelve solo los resultados, no los movimientos. Tiene tres tarjetas de indicadores, dos gráficas hechas con Chart.js y la tabla de productos por reponer, que es el detalle de la primera métrica.

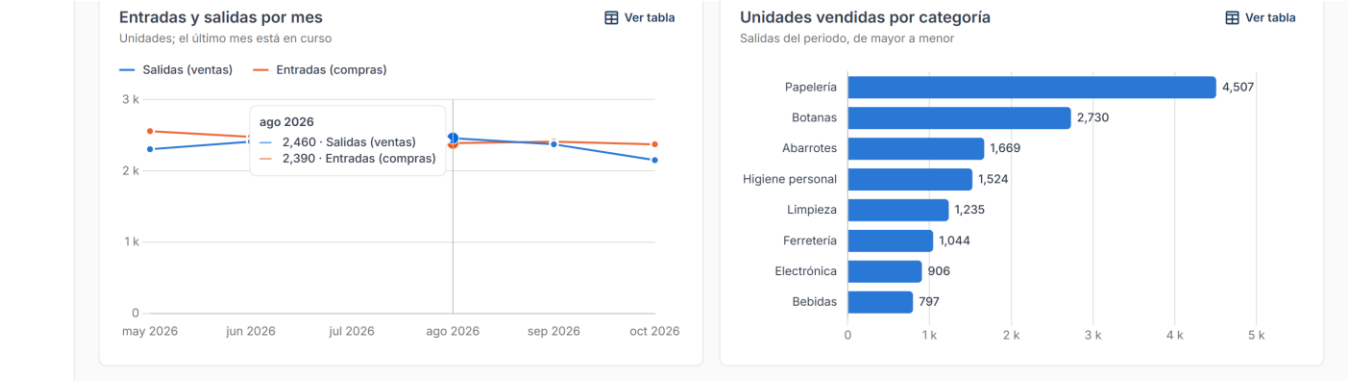
1. **Entradas y salidas por mes (línea):** compara lo que se compra contra lo que se vende cada mes. Si las salidas superan a las entradas varios meses seguidos, el inventario se está agotando.
2. **Unidades vendidas por categoría (barras):** ordena las categorías de mayor a menor venta en el periodo elegido.

Las gráficas son dinámicas: el selector de periodo recalcula las tarjetas y las gráficas, al pasar el puntero se muestran los valores exactos de cada mes o categoría, y el botón Ver tabla presenta los mismos datos como tabla. Los colores de las series se validaron para que se distingan también con daltonismo.

Dashboard con el periodo de los últimos 6 meses.



Detalle al pasar el puntero sobre agosto de 2026.



3. Consulta de referencia y método de medición

Se eligió como consulta de referencia el listado de movimientos (GET /api/movimientos), porque es la que más se ejecuta: se carga al abrir la pantalla de movimientos y se repite con cada búsqueda, cambio de filtro o cambio de página. Para cada petición la API ejecuta dos consultas en paralelo: la página de 20 registros, ordenada por fecha, y el conteo total de coincidencias para la paginación. Se midieron tres escenarios frecuentes:

1. Primera página sin filtros (lo que ve el usuario al entrar).
2. Búsqueda "cuaderno" con filtro de tipo salida.
3. Filtro por la categoría Papelería, página 10.

La prueba se automatizó con el script etl/src/benchmark.ts (npm run bench). En cada escenario hace 5 peticiones de calentamiento que no se cuentan y luego 30 peticiones medidas, una tras otra, en la misma computadora y con la misma base de datos (PostgreSQL 16 en Docker, 10,000 movimientos). Registra dos tiempos: el de las consultas en PostgreSQL, que la API reporta en el encabezado Server-Timing, y el de la petición HTTP completa. Los resultados de cada corrida quedan en docs/tarea3/resultados del repositorio.

4. Optimización aplicada

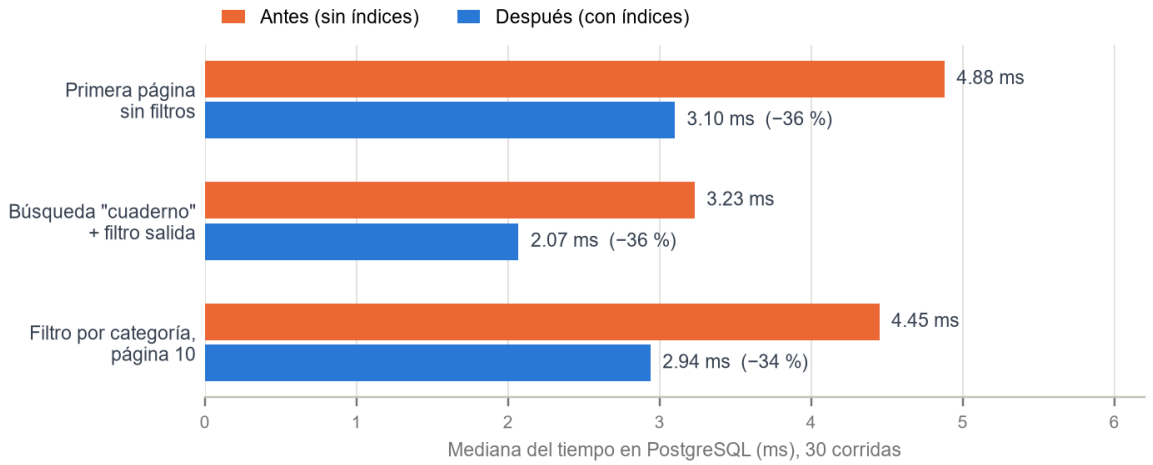
Sin índices secundarios, para devolver 20 movimientos PostgreSQL leía los 10,000, los unía con productos y usuarios y los ordenaba por fecha (Seq Scan + Sort en el plan de ejecución). Se aplicó la técnica de índice con dos índices sobre la tabla movimientos (db/indices.sql, npm run indices):

```
CREATE INDEX movimientos_fecha_idx
    ON movimientos (fecha_movimiento DESC, id_movimiento DESC);
CREATE INDEX movimientos_producto_fecha_idx
    ON movimientos (id_producto, fecha_movimiento DESC);
ANALYZE movimientos;
```

El primero entrega los movimientos ya ordenados, así que la consulta se detiene al juntar los 20 de la página. El segundo permite llegar directo a los movimientos de los productos encontrados en una búsqueda, porque PostgreSQL no indexa por su cuenta las llaves foráneas. Con EXPLAIN ANALYZE, la consulta de la primera página pasó de 4.02 ms (Seq Scan sobre 10,000 filas y ordenamiento) a 0.21 ms (Index Scan que lee solo 20 filas).

5. Resultados antes y después

Escenario	Mediana BD antes	Mediana BD después	Promedio BD antes / después	Mediana HTTP antes / después	Mejora (mediana BD)
Primera página sin filtros	4.88 ms	3.10 ms	4.88 / 3.20 ms	6.10 / 4.54 ms	36.5 %
Búsqueda "cuaderno" + filtro salida	3.23 ms	2.07 ms	3.25 / 2.12 ms	3.99 / 3.20 ms	35.9 %
Filtro por categoría, página 10	4.45 ms	2.94 ms	4.49 / 3.04 ms	5.29 / 3.74 ms	33.9 %



5.1. Interpretación

Los índices redujeron entre 34 y 37 % la mediana del tiempo en la base de datos en los tres escenarios, y entre 20 y 29 % el tiempo total de la petición HTTP. El promedio se movió igual que la mediana, así que la mejora no depende de unas cuantas corridas aisladas.

La mejora es menor que la de la consulta de la página por sí sola (de 4.02 a 0.21 ms) porque la API también ejecuta el conteo total, y ese conteo sigue recorriendo los 10,000 movimientos con sus uniones: midió 3.2 ms después de crear los índices. Como las dos consultas corren en paralelo, el conteo es ahora el que define el tiempo de respuesta. En valores absolutos la diferencia es de 1 a 2 ms, imperceptible para el usuario con 10,000 registros; lo que sí muestra la medición es que la página ya no depende del tamaño de la tabla, mientras que el conteo exacto sí. Esa es la parte que habría que cambiar al crecer.

6. Recomendaciones para 1 millón de registros

Con 1 millón de movimientos (100 veces el volumen actual) y más usuarios simultáneos, el conteo exacto, la paginación con OFFSET y las agregaciones del dashboard recorrerían cada vez más filas por petición. Se proponen tres medidas:

1. **Paginación sin conteo exacto ni OFFSET.** Mostrar un total aproximado (las estadísticas que PostgreSQL ya guarda) o guardarlo en caché unos segundos por combinación de filtros, y paginar con keyset (WHERE (fecha_movimiento, id_movimiento) < último valor visto LIMIT 20), que aprovecha el índice creado y tarda lo mismo en la página 1 que en la 5,000.
2. **Indicadores preagregados y en caché.** Guardar los totales por mes, producto y categoría en una vista materializada que se actualice de forma periódica, y guardar la respuesta del dashboard en caché (por ejemplo, Redis) por unos minutos, para que abrir el dashboard no recorra un millón de filas en cada visita.
3. **Infraestructura preparada para crecer.** Usar PostgreSQL administrado con una réplica de solo lectura para el dashboard y los reportes, un pool de conexiones (PgBouncer) y varias instancias de la API detrás de un balanceador; además, particionar la tabla de movimientos por mes para que las consultas por periodo solo lean las particiones necesarias.

Conclusión

El dashboard convierte los movimientos en tres indicadores que apoyan decisiones concretas: qué reponer, cuánto comprar y qué categorías priorizar. La prueba comparativa, con 30 corridas por escenario antes y después, mostró que los índices bajan entre 34 y 37 % el tiempo de la consulta de referencia en la base de datos, y que el siguiente cuello de botella es el conteo exacto de registros.

Los 10,000 registros simulan el crecimiento de un negocio, pero no convierten a StockAlert en una plataforma de Big Data: con este volumen, la mediana de todas las peticiones medidas quedó por debajo de 7 ms. Las conclusiones se basan en lo que se midió; las recomendaciones para 1 millón de registros son la ruta propuesta para cuando el volumen y el número de usuarios lo requieran, y tendrían que validarse con mediciones a esa escala.